

HTML & CSS Assignment 2

Question 1: Zomato-style Restaurant Card (Box Model)

Create an HTML file that displays a Zomato-style restaurant card with a restaurant name, cuisine type, and rating. Use CSS to add margin, padding, and a visible border to the card so that each section is clearly separated.

restaurant-card.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Zomato Style Restaurant Card</title>
  <style>
    .card {
      width: 280px;
      margin: 20px;
      padding: 16px;
      border: 1px solid #e0e0e0;
      border-radius: 8px;
      font-family: Arial, sans-serif;
      box-shadow: 0 2px 5px rgba(0,0,0,0.08);
    }

    .name {
      margin: 0 0 8px 0;
      padding: 4px 0;
      border-bottom: 1px solid #eee;
      font-size: 20px;
      font-weight: bold;
      color: #1c1c1c;
    }

    .cuisine {
      margin: 8px 0;
      padding: 6px 0;
      border-bottom: 1px solid #eee;
      color: #696969;
      font-size: 14px;
    }

    .rating {
      margin-top: 8px;
      padding: 6px 10px;
      border: 1px solid #267e3e;
      border-radius: 4px;
      display: inline-block;
      color: #267e3e;
      font-weight: bold;
    }
  </style>
</head>
<body>

  <div class="card">
    <div class="name">Spice Villa</div>
    <div class="cuisine">North Indian, Chinese, Mughlai</div>
```

```
<div class="rating">★ 4.3</div>
</div>

</body>
</html>
```

Explanation: Each section of the card (name, cuisine, rating) has its own `margin`, `padding`, and a `border-bottom` or `full border` to keep it visually separated from the next section. The outer `.card` container itself has `margin` to space it from other cards, `padding` to keep content away from its edges, and a visible `border` to frame the whole card.

Question 2: Flipkart-style Product Tile (Display Properties)

Design a Flipkart-style product tile with an image, product name, and price. Use different display properties (`block`, `inline-block`, `none`) to control the layout of the product name and price, and hide the price when a specific CSS class is added.

product-tile.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Flipkart Style Product Tile</title>
  <style>
    .tile {
      width: 220px;
      padding: 12px;
      border: 1px solid #ddd;
      border-radius: 6px;
      font-family: Arial, sans-serif;
    }

    .tile img {
      width: 100%;
      display: block;          /* image takes its own full-width line */
    }

    .product-name {
      display: block;          /* forces name onto its own line */
      margin-top: 10px;
      font-size: 15px;
      color: #212121;
    }

    .price {
      display: inline-block; /* sits inline but keeps box properties */
      margin-top: 6px;
      padding: 3px 8px;
      background-color: #f0fff4;
      color: #388e3c;
      font-weight: bold;
      border-radius: 4px;
    }

    /* When this class is added to .price, the price is hidden */
    .price-hidden {
      display: none;
    }
  </style>
</head>
```

```

<body>

  <div class="tile">
    
    <span class="product-name">Men's Running Shoes</span>

    <!-- Add class="price price-hidden" to hide the price -->
    <span class="price">₹1,299</span>
  </div>

</body>
</html>

```

Explanation: The product image and name use `display: block` so each one occupies its own full line, stacking the tile vertically. The price uses `display: inline-block`, which lets it sit compactly while still accepting padding, background, and border-radius. To hide the price, the extra class `price-hidden` (with `display: none`) is added to the same element, e.g. `class="price price-hidden"` — this removes the price from the layout completely, unlike `visibility: hidden`, which would leave empty space behind.

Question 3: Fixed WhatsApp Chat Notification Badge

Build a fixed-position WhatsApp chat notification badge at the bottom right corner of the screen. Use `position: fixed` and appropriate margin and padding to ensure it stays visible even when scrolling.

whatsapp-badge.html

```

<!DOCTYPE html>
<html>
<head>
  <title>WhatsApp Chat Badge</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      height: 1500px; /* tall page to demonstrate scrolling */
    }

    .whatsapp-badge {
      position: fixed;
      bottom: 25px;
      right: 25px;
      margin: 0;
      padding: 14px;
      background-color: #25d366;
      color: #ffffff;
      border-radius: 50%;
      width: 30px;
      height: 30px;
      text-align: center;
      font-size: 20px;
      box-shadow: 0 3px 8px rgba(0,0,0,0.3);
      cursor: pointer;
    }
  </style>
</head>
<body>

  <h1>Scroll down to test the fixed badge</h1>

```

```
<div class="whatsapp-badge">📞</div>

</body>
</html>
```

Explanation: `position: fixed` takes the badge out of the normal document flow and anchors it relative to the browser viewport, not the page content. Because of this, `bottom: 25px` and `right: 25px` keep it pinned to the bottom-right corner at all times, even as the user scrolls through a tall page. `padding` keeps the icon centered and gives the circular badge its size, while the `border-radius` of 50% makes it round like WhatsApp's real chat bubble.

Question 4: BookMyShow Poster Grid with 'New' Badge (Relative/Absolute Positioning)

Create a BookMyShow movie poster grid where each poster uses `position: relative`. Add a 'New' badge to the top-left corner of one poster using `position: absolute`, so it overlays the image correctly.

poster-grid.html

```
<!DOCTYPE html>
<html>
<head>
  <title>BookMyShow Poster Grid</title>
  <style>
    .poster-grid {
      display: flex;
      gap: 16px;
      flex-wrap: wrap;
      font-family: Arial, sans-serif;
    }

    .poster {
      position: relative; /* establishes positioning context for the badge */
      width: 160px;
    }

    .poster img {
      width: 100%;
      border-radius: 6px;
      display: block;
    }

    .new-badge {
      position: absolute;
      top: 8px;
      left: 8px;
      background-color: #c4242b;
      color: #ffffff;
      padding: 3px 8px;
      font-size: 12px;
      font-weight: bold;
      border-radius: 4px;
    }
  </style>
</head>
<body>

  <div class="poster-grid">
```

```

<div class="poster">
  <span class="new-badge">NEW</span>
  
</div>

<div class="poster">
  
</div>

<div class="poster">
  
</div>

</div>

</body>
</html>

```

Explanation: Each `.poster` container is given `position: relative`, which does not move it visually but makes it the reference point for any absolutely positioned children inside it. The `.new-badge` span uses `position: absolute` with `top: 8px; left: 8px;`, so it is placed 8px from the top-left corner of its nearest positioned ancestor — the poster div — rather than the whole page. This makes the badge overlay neatly on top of that one poster's image only.

Question 5: Refactoring with a Reusable CSS Class

Refactor your CSS for any one of the above tasks to reduce repeated margin and padding values by using a reusable CSS class. Explain in a comment how this improves maintainability.

Refactoring Question 1 (the Zomato-style restaurant card), where `.name`, `.cuisine`, and `.rating` originally repeated similar margin/padding patterns:

restaurant-card-refactored.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Zomato Card - Refactored</title>
  <style>
    .card {
      width: 280px;
      margin: 20px;
      padding: 16px;
      border: 1px solid #e0e0e0;
      border-radius: 8px;
      font-family: Arial, sans-serif;
      box-shadow: 0 2px 5px rgba(0,0,0,0.08);
    }

    /*
     Reusable utility class "card-section":
     Instead of writing margin, padding, and border-bottom
     separately inside .name and .cuisine, we define them once
     here and simply attach this class to any element that needs
     the same spacing pattern. This reduces duplicate code,
     makes the spacing consistent across all sections, and means
     that if the design changes (e.g. more padding is needed),
     we only update ONE rule instead of editing every section
     individually. This is easier to maintain and less error-prone
     as the project grows.

```

```

    */
    .card-section {
        margin: 8px 0;
        padding: 6px 0;
        border-bottom: 1px solid #eee;
    }

    .name {
        font-size: 20px;
        font-weight: bold;
        color: #1c1c1c;
    }

    .cuisine {
        color: #696969;
        font-size: 14px;
    }

    .rating {
        margin-top: 8px;
        padding: 6px 10px;
        border: 1px solid #267e3e;
        border-radius: 4px;
        display: inline-block;
        color: #267e3e;
        font-weight: bold;
    }
</style>
</head>
<body>

<div class="card">
    <div class="card-section name">Spice Villa</div>
    <div class="card-section cuisine">North Indian, Chinese, Mughlai</div>
    <div class="rating">★ 4.3</div>
</div>

</body>
</html>

```

Explanation: The repeated `margin: 8px 0; padding: 6px 0; border-bottom: 1px solid #eee;` that both `.name` and `.cuisine` used has been pulled out into one reusable class, `.card-section`. Both elements now include `class="card-section name"` and `class="card-section cuisine"`, combining the shared spacing rule with their own unique styling (font size, color). This follows the DRY (Don't Repeat Yourself) principle — if the spacing needs to change later, it is updated in a single place instead of in every individual section, reducing the chance of inconsistency and making the CSS easier to scale and maintain.